

Project 1: Content-based Recommendation

추천시스템 · 2025-27567 배문성 · 2026년 9월

1. 전체 구조

main.py는 데이터 (load_game_data, load_ratings_data) → representation 생성 (build_semantic_matrix, build_tfidf_matrix) → 추천·예측 (ItemCF, WeightedItemCF) → 출력(write_output) 순서로 동작한다. representation만 교체하면 동일한 로직을 쓸 수 있으므로 Item-based CF는 ItemCF 클래스 하나로 구현하고 semantics용·TF-IDF용 인스턴스를 각각 생성했다.

ratings.csv의 game_id는 원본 ID(예: 282440)이므로 build_game_index로 game_id → 행렬 row index 매핑을 만들어, 행렬에 접근하기 전에 반드시 이 매핑을 거치도록 했다.

2. Description 기반 representation (Task 1)

- load_game_data(): games_metadata.json을 한 줄씩 json.loads로 파싱한 뒤 game_id 기준으로 정렬하여 game_ids, descriptions, tags를 같은 순서로 반환한다. 이후 만드는 모든 행렬의 행 순서가 이 정렬 순서와 일치한다.
- build_semantic_matrix(): SentenceTransformer("all-MiniLM-L6-v2")로 description을 일괄 인코딩해 shape [19755, 384] 행렬을 만든다. 인코딩에 수 분이 걸리므로 결과를 cache/semantic_matrix.npy에 저장하고, 캐시의 행 수가 description 수와 일치할 때만 재사용한다.

3. Tag 기반 TF-IDF representation (Task 2)

build_tfidf_matrix()에서 전체 tag를 모아 alphabetical order로 정렬해 열 순서를 고정한다 (unique tag 440개). 이후

- IDF**: Counter로 각 tag가 등장한 game 수를 세고 $\text{np.log10}(\text{total_count} / \text{tag_count})$ 로 계산.
- TF**: game 하나에 대해 $n(d, t)/n(d)$, 즉 해당 tag의 등록 횟수를 그 game의 전체 tag 수로 나눈 값.
- 각 game이 실제로 가진 tag만 순회하며 $\text{tfidf_matrix}[\text{row}, \text{tag_col}[\text{tag}]] += \text{idf}[\text{tag}] / n_d$ 로 누적해 shape [19755, 440] 행렬을 완성한다. tag가 없는 game은 0 벡터로 남는다.

4. Cosine similarity (Task 3)

ItemCF.__init__에서 representation을 행 단위로 L2 정규화한다($V / \max(\|V\|, \epsilon)$). 이렇게 두면 이후의 모든 내적이 곧 cosine similarity가 되므로, [19755, 19755] 유사도 행렬(약 1.5 GiB)을 실제로 만들지 않고도 필요한 유사도를 그때그때 얻을 수 있다. 분모의 ϵ 은 0 벡터(tag가 없는 game)로 인한 0 나눗셈을 막는다.

5. 예측 평점 계산 (Task 4)

5.1 user 이력 추출 (4-a)

UserHistory는 user 815,006명 × game 19,755개 행렬(약 103 GiB, 99.99%가 빈 칸)을 만들지 않는다. 대신 (row, rating)을 user_id로 정렬한 테이블 하나로 보관하고, 요청받은 user의 구간만 꺼내 쓴다(get()). metadata에 없는 game_id는 representation이 없으므로 이 단계에서 제외한다.

5.2 Content 기반 Item-based CF (4-b)

ItemCF.predict_all(user_id)는 해당 user가 전체 game에 남길 것으로 추정되는 점수(shape [19755])를 다음과 같이 계산한다.

$$\hat{r}(u, i) = \frac{\sum_{j \in I_u} r_{uj} \cdot \text{sim}(j, i)}{\sum_{j \in I_u} \text{sim}(j, i) + \epsilon}$$

분자·분모 모두 sim에 대해 선형이므로, [n, 19755] 유사도 행렬을 만들지 않고 $v @ (\text{ratings} @ V[\text{rows}])$ 와 $v @ V[\text{rows}].\text{sum}(\text{axis}=0)$ 처럼 벡터 하나로 접을 수 있다. 근사가 아니라 수학적으로 동일한 값이며, user당 행렬곱 두 번으로 끝난다.

recommend()는 이미 interact한 game의 점수를 -inf로 만들어 추천 대상에서 제외한 뒤 np.argsort(-scores, kind="stable")로 상위 20개를 고른다. game_ids가 오름차순이므로 stable sort를 쓰면 동점일 때 game_id 오름차순이라는 출력 요구사항이 자동으로 만족된다. 평가 이력이 없는 user에 대해서는 0 벡터를 반환한다.

5.3 가중합과 대량 쌍 예측 (4-c)

WeightedItemCF는 두 ItemCF의 예측을 $\text{Score} = \alpha \cdot \text{Score}_{\text{sem}} + (1 - \alpha) \cdot \text{Score}_{\text{tfidf}}$ 로 합치며, predict()가 쌍 하나의 점수를 반환한다.

predict_pairs()는 검증용으로 ratings.csv 전체(약 106만 쌍) 같은 대량 입력을 처리하기 위한 경로다. predict()를 반복하면 쌍마다 [19755, d] 행렬곱이 필요해 사실상 끝나지 않으므로, 쌍마다 그 user의 이력만 펼쳐 np.einsum으로 내적하고 np.bincount로 쌍 단위로 되합친다 (연산량 $\sum_u n_u^2 \approx 3.1 \text{ M}$). searchsorted로 각 쌍의 user 구간을 한 번에 찾고, 메모리를 고려해 쌍을 20,000개 단위로 나눠 처리한다. 값은 predict()를 반복한 것과 동일하다. score_rmse()는 이를 이용해 실제 rating과의 RMSE를 계산한다.

5.4 가중치 α 최적값

TODO: ratings.csv를 train/validation으로 분할하고 score_rmse()로 α 를 탐색한 결과(최적값, 탐색 방법, 0.5 대비 RMSE 개선)를 기술. 현재 코드의 ALPHA = 0.5도 탐색 결과로 교체할 것.

6. 입력 처리와 출력 포맷

`read_recommendation_input()`은 `user_id` 목록을, `read_prediction_input()`은 `user_id;game_id` 쌍을 읽는다. `semantics()`·`tfidf()`는 user당 20줄을, `weighted()`는 쌍당 1줄을 "{},{},{:.4f}" 포맷(공백 없는 콤마, 소수점 4자리 반올림)으로 만들고, `write_output()`이 `results/` 폴더를 생성해 `semantics_output.txt`, `tfidf_output.txt`, `score_prediction_output.txt`로 저장한다.

7. 어려웠던 점과 해결 방법

TODO

8. 느낀 점

TODO